

DeepSeek Models

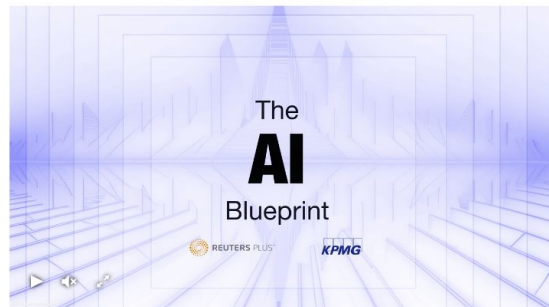
Hui Yang

The Impact of DeepSeek on the Capital Market

DeepSeek sparks AI stock selloff; Nvidia posts record market-cap loss

By Sinéad Carew, Amanda Cooper and Ankur Banerjee

January 27, 2025 4:31 PM PST - Updated 15 days ago



Summary Companies

- Tech stocks sink, Nvidia drops sharply
- China's DeepSeek AI assistance surges in popularity
- Nervous investors seek safe havens, dollar falls

Business / Tech

A shocking Chinese AI advancement called DeepSeek is sending US stocks plunging



By David Goldman and Matt Egan, CNN

3 minute read · Updated 4:21 PM EST, Mon January 27, 2025

Facebook X Email Print 541 comments



Video Ad Feedback

The New York Times

China's A.I. Advances Spook Big Tech Investors on Wall Street

Advances in artificial intelligence by Chinese upstarts rattled U.S. markets on Monday, with the threat of greater competition prompting a...

2 weeks ago

NEWS > MARKETS NEWS

Here Are the Stock Market Winners and Losers from the DeepSeek AI Upheaval

By COLIN LAIDLEY Published February 01, 2025 07:00 AM EST



Bryan R. Smith / AFP / Getty Images

Bloomberg

Nvidia's \$589 Billion DeepSeek Rout Is Largest in Market History

Recent Western News Focus on US-China Competition

What DeepSeek's Success Says About China's Ability to Nurture Talent

China produces a vast number of STEM graduates, but it hasn't been known for innovation. Cultural and political factors may help explain why.

▶ Listen to this article - 7:34 min [Learn more](#)

📖 Share full article



💬 116



A student preparing for the 2025 national postgraduate entrance exam at Hebei University of Engineering in Handan, China, in December. Costfoto/NurPhoto, via Getty Images

DeepSeek banned from government devices in New York state

DeepSeek has the capability to send user data to the Chinese government.

By [Aaron Katersky](#)

February 10, 2025, 10:19 AM



CONDOLEEEZZA RICE, AMY ZEGART: China's DeepSeek AI escalates fight to innovate. 4 trends we don't dare miss

4 key trends for innovation include speed of emerging technologies and erosion of US leadership

By [Condoleezza Rice](#), [Amy Zegart](#) **OPINION** Fox News

[China technology](#) [Opinion / China Opinion](#)

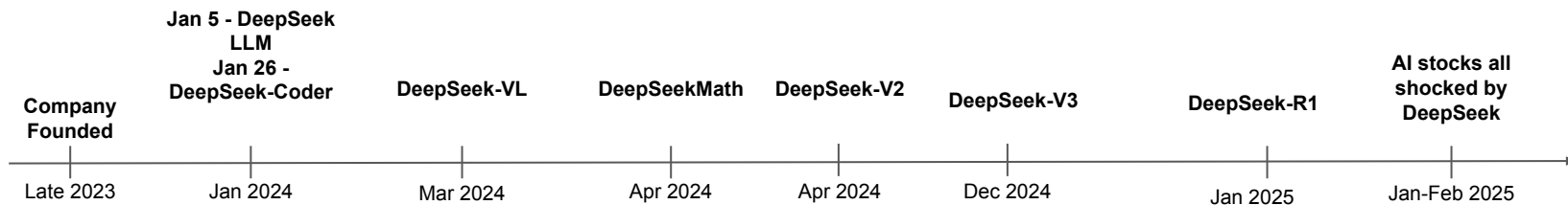


Zhou Xin

My Take | Why DeepSeek's true value for China goes beyond AI advance

DeepSeek has far-reaching implications for China's tech ambitions beyond its immediate achievements

DeepSeek Timeline - Staggeringly Fast Iterated



Index

- **Early Models**
 - **DeepSeekLLM (v1)**
 - **DeepSeek-Coder**
 - **DeepSeek-VL**
 - **DeepSeek-Math**
- **Late Models**
 - DeepSeek-V2
 - DeepSeek-V3
 - DeepSeek-R1

DeepSeek LLM

- Size: 7B and 67B
- 2T tokens
- Alignment Techniques
 - Supervised fine-tuning, Direct Preference Optimization
- Performance
 - Beat LLaMA-2 70B on a range of benchmarks
 - Especially good at domains of code
- Infra
 - HAL-LLM framework
 - Train the model in FP16 but accumulate gradients in FP32

DeepSeek LLM - Tech Spec & Innovations

- Aggressive Pre-training Deduplication
 - Focusing on increasing the presence of underrepresented domains

Dumps Used	1	2	6	12	16	22	41	91
Deduplication Rate (%)	22.2	46.7	55.7	69.9	75.7	76.3	81.6	89.8

Table 1 | Deduplication ratios for various Common Crawl dumps.

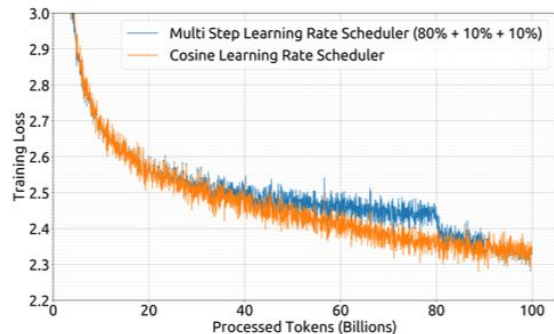
- Architecture

Params	n_{layers}	d_{model}	n_{heads}	$n_{\text{kv_heads}}$	Context Length	Sequence Batch Size	Learning Rate	Tokens
7B	30	4096	32	32	4096	2304	4.2e-4	2.0T
67B	95	8192	64	8	4096	4608	3.2e-4	2.0T

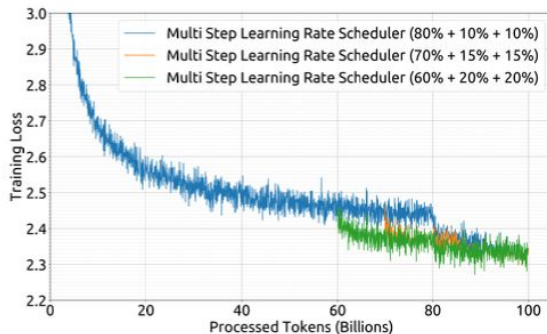
Table 2 | Detailed specs of DeepSeek LLM family of models. We choose the hyper-parameters based on our findings in Section [3](#)

DeepSeek LLM - Tech Spec & Innovations

- Multi-step Learning Rate could work better than Cosine if tuned



(a) Multi-step v.s. cosine learning rate decay



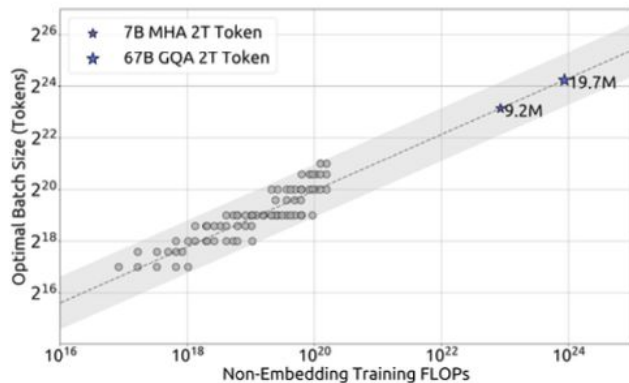
(b) Different proportions of multi-step stages

Figure 1 | Training loss curves with different learning rate schedulers or different parameters for schedulers. The model size is 1.6 billion parameters, trained on a dataset of 100 billion tokens.

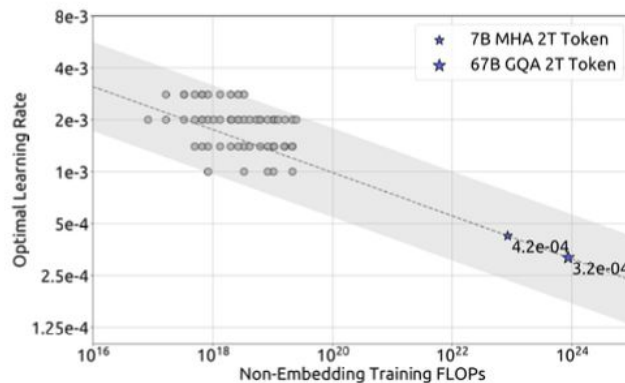
A multi-step learning rate scheduler is employed during pre-training instead of the typical cosine scheduler. Specifically, the learning rate of the model reaches its maximum value after 2000 warmup steps, and then decreases to 31.6% of the maximum value after processing 80% of the training tokens. It further reduces to 10% of the maximum value after 90% of the tokens. The gradient clipping during the training phase is set to 1.0.

DeepSeek LLM - Scaling Law

- Optimal Batch Size and Learning Rate correlates with training FLOPs



(a) Batch size scaling curve



(b) Learning rate scaling curve

Figure 3 | Scaling curves of batch size and learning rate. The grey circles represent models whose generalization error exceeded the minimum by no more than 0.25%. The dotted line represents the power law fitting the smaller model. The blue stars represent DeepSeek LLM 7B and 67B.

DeepSeek LLM - Performance

Language	Benchmark	Test-shots	LLaMA2 7B	DeepSeek 7B	LLaMA2 70B	DeepSeek 67B
English	HellaSwag	0-shot	75.6	75.4	84.0	84.0
	PIQA	0-shot	78.0	79.2	82.0	83.6
	WinoGrande	0-shot	69.6	70.5	80.4	79.8
	RACE-Middle	5-shot	60.7	63.2	70.1	69.9
	RACE-High	5-shot	45.8	46.5	54.3	50.7
	TriviaQA	5-shot	63.8	59.7	79.5	78.9
	NaturalQuestions	5-shot	25.5	22.2	36.1	36.6
	MMLU	5-shot	45.8	48.2	69.0	71.3
	ARC-Easy	0-shot	69.1	67.9	76.5	76.9
	ARC-Challenge	0-shot	49.0	48.1	59.5	59.0
	OpenBookQA	0-shot	57.4	55.8	60.4	60.2
	DROP	1-shot	39.8	41.0	69.2	67.9
	MATH	4-shot	2.5	6.0	13.5	18.7
	GSM8K	8-shot	15.5	17.4	58.4	63.4
	HumanEval	0-shot	14.6	26.2	28.7	42.7
	MBPP	3-shot	21.8	39.0	45.6	57.4
	BBH	3-shot	38.5	39.5	62.9	68.7
	AGIEval	0-shot	22.8	26.4	37.2	41.3
	Pile-test	-	0.741	0.725	0.649	0.642
Chinese	CLUEWSC	5-shot	64.0	73.1	76.5	81.0
	CHID	0-shot	37.9	89.3	55.5	92.1
	C-Eval	5-shot	33.9	45.0	51.4	66.1
	CMMLU	5-shot	32.6	47.2	53.1	70.8
	CMath	3-shot	25.1	34.5	53.9	63.0
	C3	0-shot	47.4	65.4	61.7	75.3
	CCPM	0-shot	60.7	76.9	66.2	88.5

DeepSeek-Coder

- Parameter Size:
 - 1.3B and 33B
- 2T Tokens
- 16K Context Window

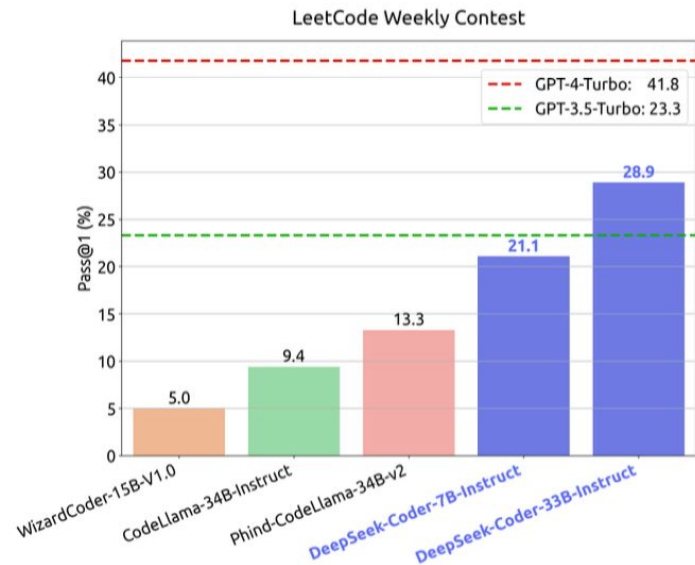
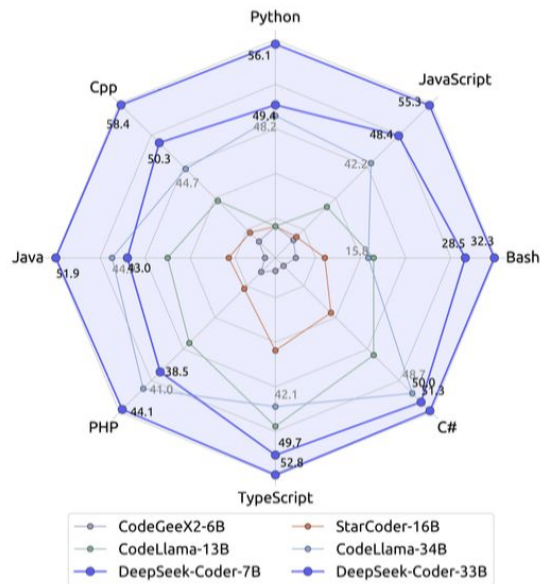


Figure 1 | The Performance of DeepSeek-Coder

DeepSeek-Coder: Innovations

- Dependency Sorting and Puts all files together as one training text
- Training Objectives: NTP & Fill In the Middle
 - PSM (Prefix-Suffix-Middle)
 - SPM (Suffix-Prefix-Middle)

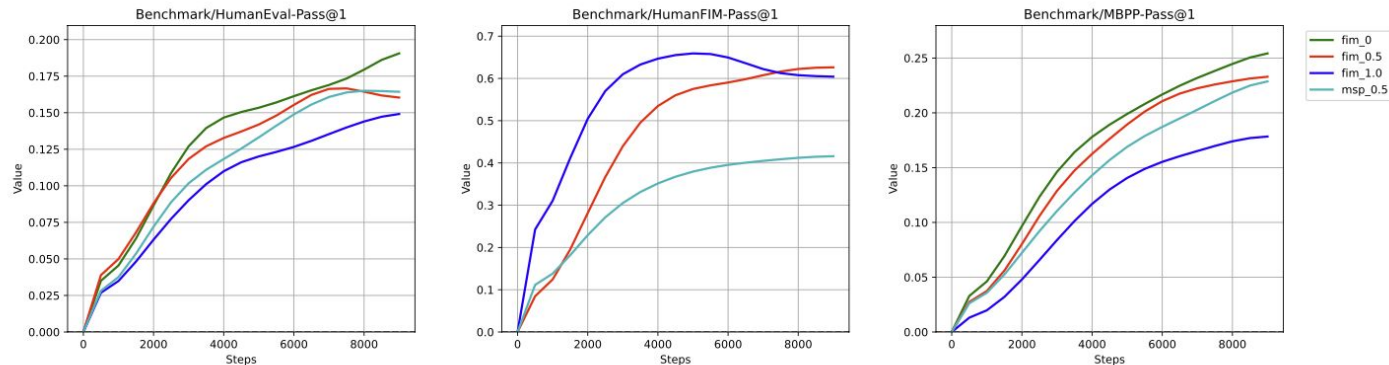


Figure 3 | The effectiveness of using FIM objective.

DeepSeek-VL

- Vision-Language Model
 - Hybrid Vision Encoder Integrated with LLM
- 1.3B and 7B
- SigLIP as the Vision encoder - CLIP family
- Two-layer hybrid MLP as the Vision-Language Adapter
- LLM using DeepSeek LLM



Prompt

Please generate the latex code of the formula in the image.

$$A_n = a_0 \left[1 + \frac{3}{4} \sum_{k=1}^n \left(\frac{4}{9} \right)^k \right]$$



Response

$A_n = a_0 \left[1 + \frac{3}{4} \sum_{k=1}^n \left(\frac{4}{9} \right)^k \right]$

DeepSeek-VL

- Pre-Training Data

Table 1 | Summary of datasets used in the joint vision and language pretraining stage.

Category	Dataset	Ratio
Interleaved image-text	MMC4 (Zhu et al., 2024)	13.1%
	Wikipedia EN& CN (Foundation)	
	Wikihow (Yang et al., 2021)	
	in-house PDF and Epub textbooks	
Image caption	Capsfusion (Yu et al., 2023a)	11.1%
	TaiSu (Liu et al., 2022b)	
	Detailed Caption (echo840, 2024)	
Table and chart	Chart2text (Kantharaj et al., 2022)	2.1%
	Geo170K (Gao et al., 2023)	
	Ureader (Ye et al., 2023)	
	Unichart (Masry et al., 2023)	
	M-paper (Hu et al., 2023)	
	ScienceQA (Lu et al., 2022b)	
	ScreenQA (Hsiao et al., 2022)	
	SciGraphQA-295K (Li and Tajbakhsh, 2023)	
	Paper2figure100k (Rodriguez et al., 2023)	
	Widget Captioning (Li et al., 2020)	
	Screen2words (Wang et al., 2021)	
Web Code	Refexp (Mao et al., 2016)	0.4%
	Websight (HuggingFaceM4, 2024)	
python plots scraped from GitHub notebook		
Scene text OCR	ArT (Chng et al., 2019)	1.2%
	MLT-17 (Nayef et al., 2017)	
	LSVT (Sun et al., 2019)	
	UberText (Zhang et al., 2017)	
	Coco-text (Veit et al., 2016)	
	RCTW-17 (Shi et al., 2017)	
	ReCTS (Zhang et al., 2019)	
	TextOCR (Singh et al., 2021)	
	OpenVINO (Krylov et al., 2021)	
	HierText (Long et al., 2022)	
Document OCR	arXiv rendered markdown (Blecher et al., 2023)	2.1%
Text-only corpus	DeepSeek-LLM 2T text copus (DeepSeek-AI, 2024)	70.0%

DeepSeek-VL: Training Pipeline

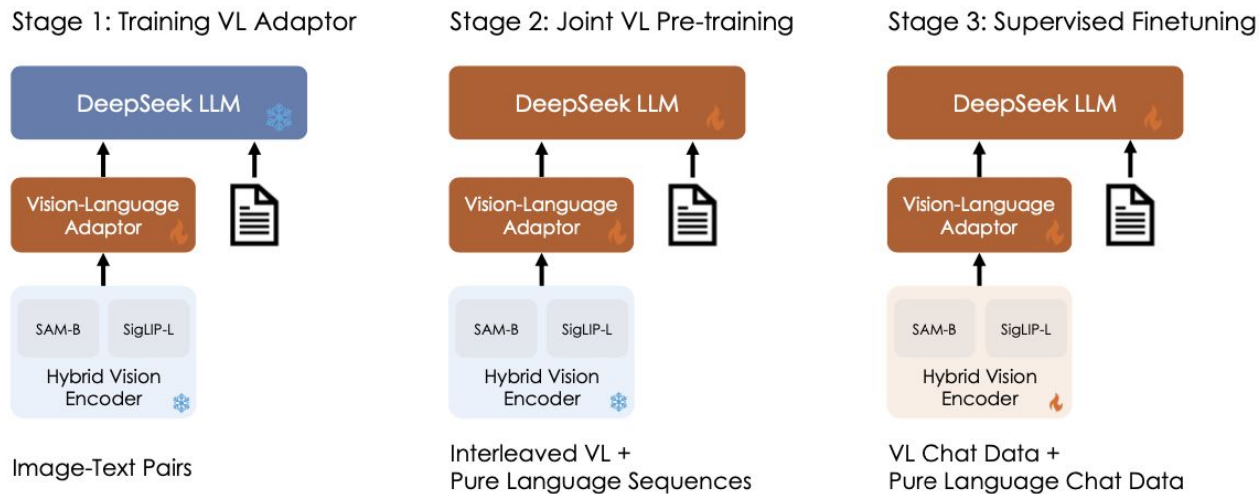
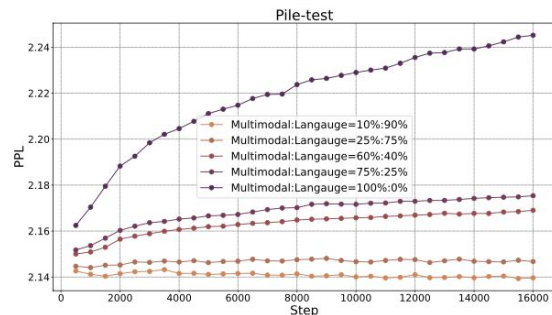
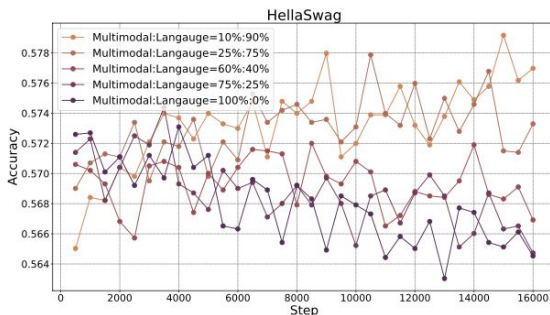
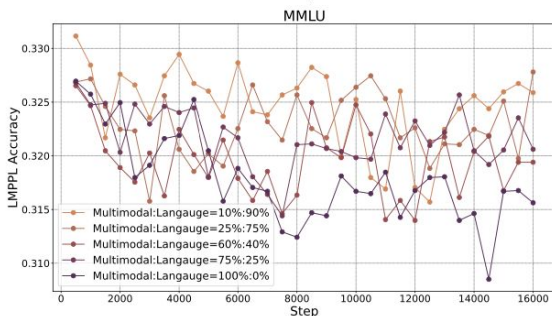
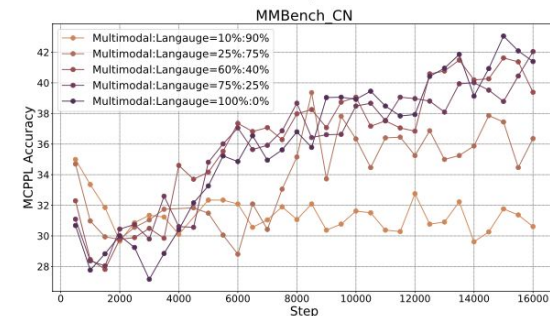
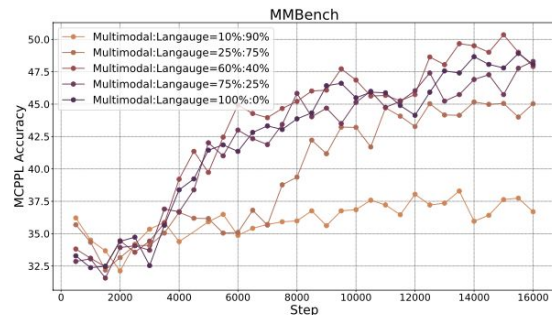
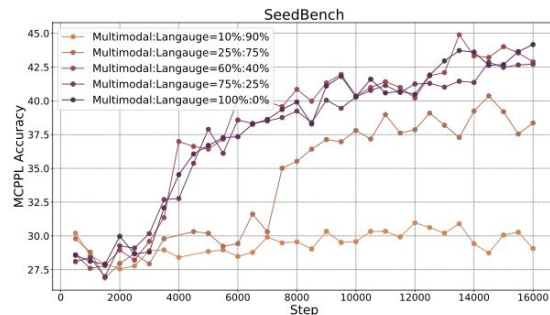


Figure 3 | Our training pipelines consist of three stages. Stage 1 involves training the Vision-Language (VL) adaptor while keeping the hybrid vision encoder and language model fixed. Stage 2 is the crucial part of the joint vision and language pretraining, where both VL adaptor and language model are trainable. Stage 3 is the supervised fine-tuning phase, during which the low-resolution vision encoder SigLIP-L, VL adaptor, and language model will be trained.

DeepSeek-VL: Right Training Mixture



Performance: Beat Open source VLMs on most tasks

	LLM	MMMU	CMMMU	MMB	MMC	SEED	OCRB	POPE	MathV	MMVet
Close-source LMMs:										
Gemini Pro	Unk	48.9	-	75.2	74.0	70.7	659	-	45.2	59.2
GPT-4V	Unk	56.8	42.5	75.0	74.7	71.6	659	-	47.8	49.9
Qwen-VL-Plus	Unk	45.2	39.5	66.2	69.6	72.7	-	-	43.3	55.7
Qwen-VL-MAX	Unk	51.4	-	78.1	76.4	72.7	-	-	51.0	61.8
Open-source 13B LMMs:										
LLaVA-1.5	13B	36.4	-	68.2	61.9	68.2	331	85.9	26.4	38.3
VILA	13B	-	-	70.3	64.3	-	-	84.2	-	38.8
LLaVA-Next	13B	36.2	-	70.0	64.4	71.9	-	86.7	35.3	48.4
Open-source 7B LMMs:										
EMU2-Chat	7B	36.3	23.8	63.6	45.9	68.9	-	-	30.0	31.0
Qwen-VL-Chat	7B	37.0	-	60.6	56.7	64.8	-	-	33.8	47.3
CogVLM	7B	37.3	24.8	63.7	53.8	68.8	-	-	34.7	54.5
LLaVA-Next	7B	35.8	-	67.4	60.0	70.2	-	86.5	34.6	43.9
Yi-VL	6B	37.8	35.8	68.2	68.9	67.6	-	-	28.0	31.1
DeepSeek-VL (ours)	7B	36.6	37.9	73.2	72.8	70.4	456	88.1	36.1	41.5

Table 5 | The comparison between different multi-modal models. The top half are proprietary models, while the bottom are open-source models.

DeepSeek Math

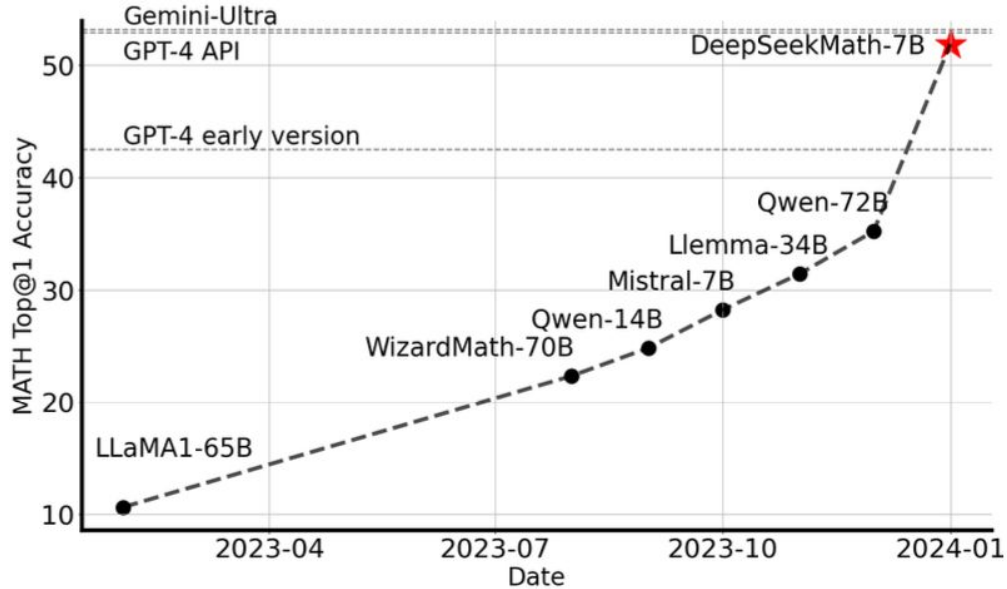


Figure 1 | Top1 accuracy of open-source models on the competition-level MATH benchmark (Hendrycks et al., 2021) without the use of external toolkits and voting techniques.

DeepSeek Math - Key Innovations

- A curated Math specific Corpus
 - Keep the top 40B Tokens based on the quality score from fastText model

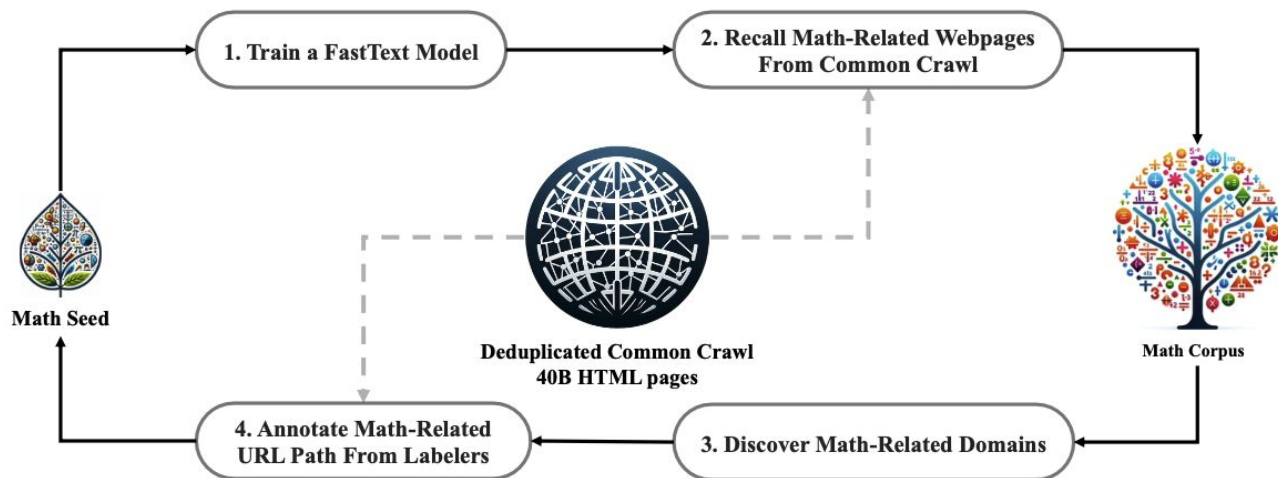


Figure 2 | An iterative pipeline that collects mathematical web pages from Common Crawl.

DeepSeek Math - Key Innovations

- A new Reinforcement Learning Algorithm
 - GPRO - Group Relative Policy Optimization

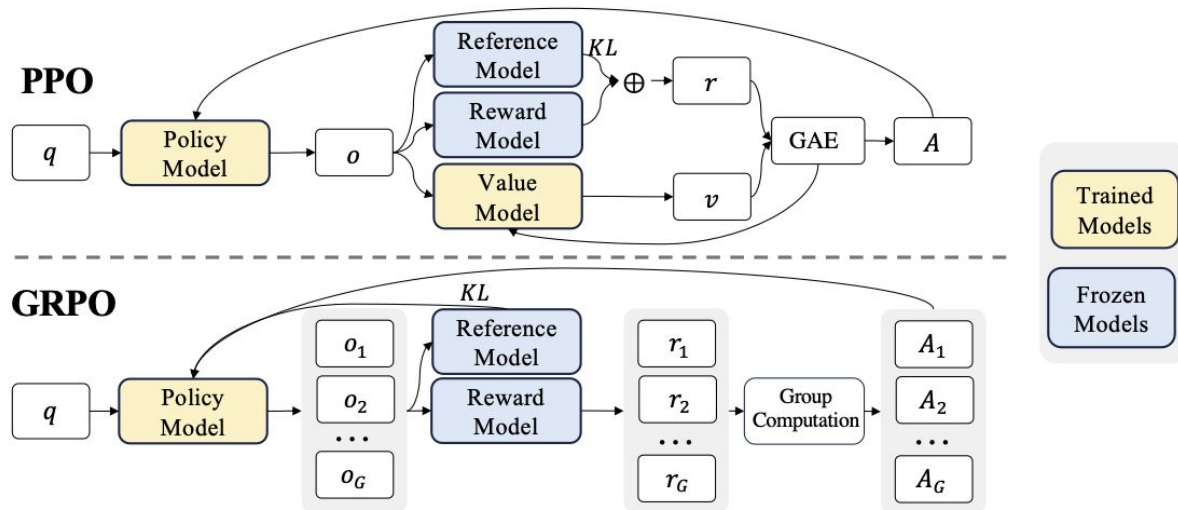


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

DeepSeek Math - Key Innovations

- A new Reinforcement Learning Algorithm
 - GPRO - Group Relative Policy Optimization
- Biggest Benefits
 - No need to frequently update Reward model
 - Optimize the general alignment directions of a group each time
 - $\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})},$
 - The KL Divergence is only added to the Loss instead the Advantage function
 - Both Outcome Supervision and Process Supervision

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}, \quad (3)$$

DeepSeek Math - Effectiveness of RL Iterations

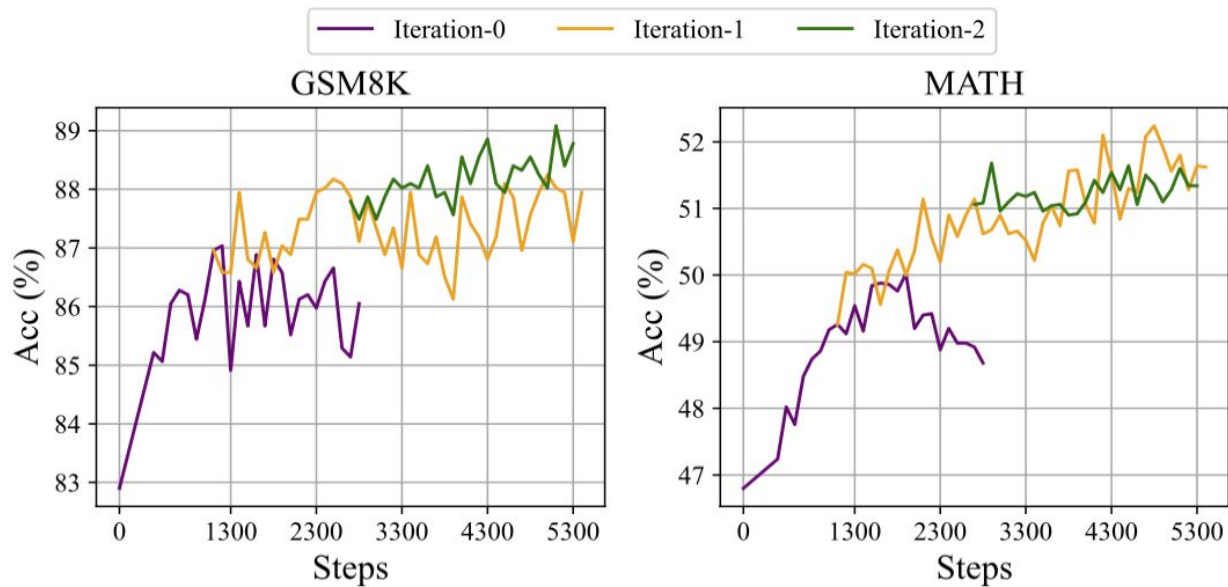


Figure 6 | Performance of iterative reinforcement learning with DeepSeekMath-Instruct 7B on two benchmarks.

DeepSeek Math - Performance

Model	Size	English Benchmarks					Chinese Benchmarks		
		GSM8K	MATH	OCW	SAT	MMLU STEM	CMATH	Gaokao MathCloze	Gaokao MathQA
Closed-Source Base Model									
Minerva	7B	16.2%	14.1%	7.7%	-	35.6%	-	-	-
Minerva	62B	52.4%	27.6%	12.0%	-	53.9%	-	-	-
Minerva	540B	58.8%	33.6%	17.6%	-	63.9%	-	-	-
Open-Source Base Model									
Mistral	7B	40.3%	14.3%	9.2%	71.9%	51.1%	44.9%	5.1%	23.4%
Llemma	7B	37.4%	18.1%	6.3%	59.4%	43.1%	43.4%	11.9%	23.6%
Llemma	34B	54.0%	25.3%	10.3%	71.9%	52.9%	56.1%	11.9%	26.2%
DeepSeekMath-Base	7B	64.2%	36.2%	15.4%	84.4%	56.5%	71.7%	20.3%	35.3%

Table 2 | Comparisons between DeepSeekMath-Base 7B and strong base models on English and Chinese mathematical benchmarks. Models are evaluated with chain-of-thought prompting. Minerva results are quoted from [Lewkowycz et al. \(2022a\)](#).

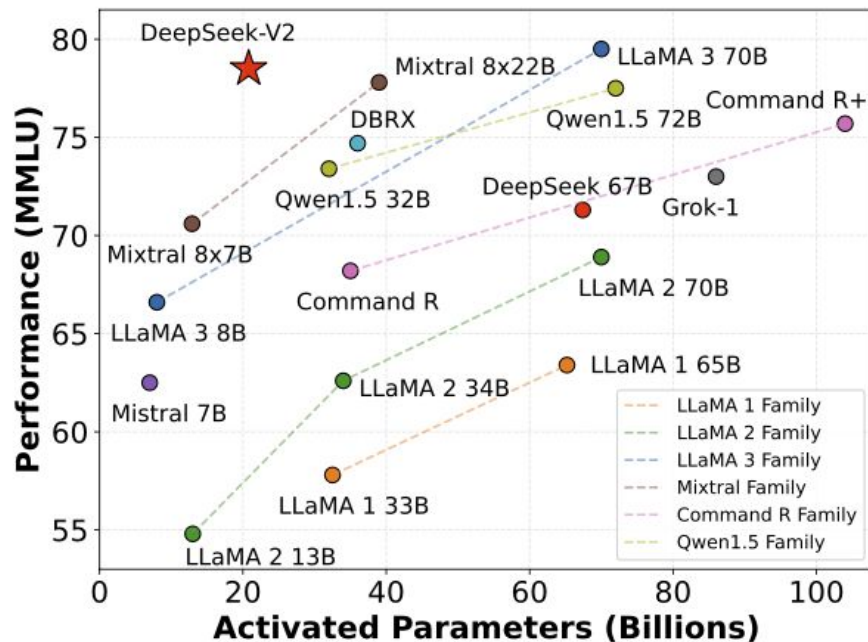
Index

- Early Models
 - 2024 H1
 - DeepSeekLLM (v1)
 - DeepSeek-Coder
 - DeepSeek-VL
 - DeepSeek-Math
- **Late Models**
 - **DeepSeek-V2**
 - **DeepSeek-V3**
 - **DeepSeek-R1**

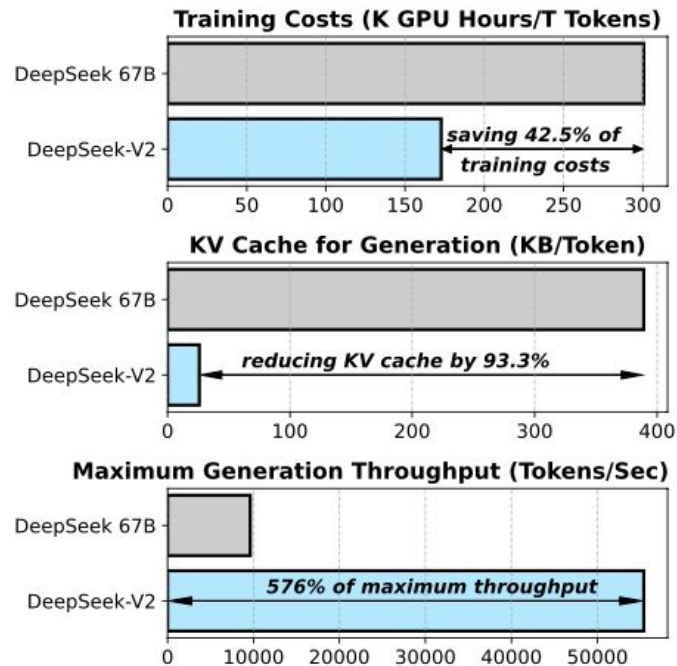
DeepSeek-V2

- Size: 236B
- MoE, so only 21B activated for each token
- 128K context length
- Multi-head Latent Attention (MLA)
 - Compared with DeepSeek 67B,
 - Saves 42.5% of training costs
 - Reduces KV cache by 93.3%
 - Boosts Generation throughput to 5.76 times
- 8.1T tokens pre-training

DeepSeek-V2

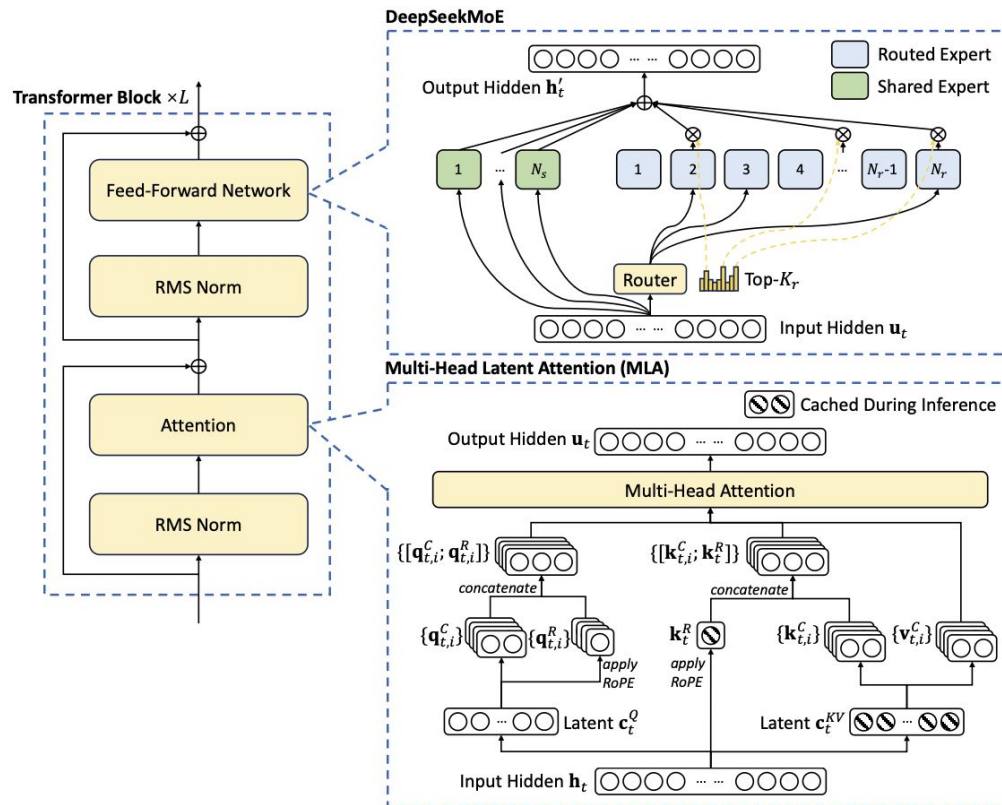


(a)



(b)

DeepSeek-V2 Architecture



DeepSeek-V2 MLA

$$\mathbf{c}_t^{KV} = W^{DKV} \mathbf{h}_t,$$

$$\mathbf{k}_t^C = W^{UK} \mathbf{c}_t^{KV},$$

$$\mathbf{v}_t^C = W^{UV} \mathbf{c}_t^{KV},$$

where $\mathbf{c}_t^{KV} \in \mathbb{R}^{d_c}$ is the compressed latent vector for keys and values; $d_c (\ll d_h n_h)$ denotes the KV compression dimension; $W^{DKV} \in \mathbb{R}^{d_c \times d}$ is the down-projection matrix; and $W^{UK}, W^{UV} \in \mathbb{R}^{d_h n_h \times d_c}$ are the up-projection matrices for keys and values, respectively. During inference, MLA only needs to cache $\mathbf{c}_t^{KV}$, so its KV cache has only $d_c l$ elements, where l denotes the number of layers. In addition, during inference, since W^{UK} can be absorbed into W^Q , and W^{UV} can be absorbed into W^O , we even do not need to compute keys and values out for attention. Figure 3 intuitively illustrates how the KV joint compression in MLA reduces the KV cache.

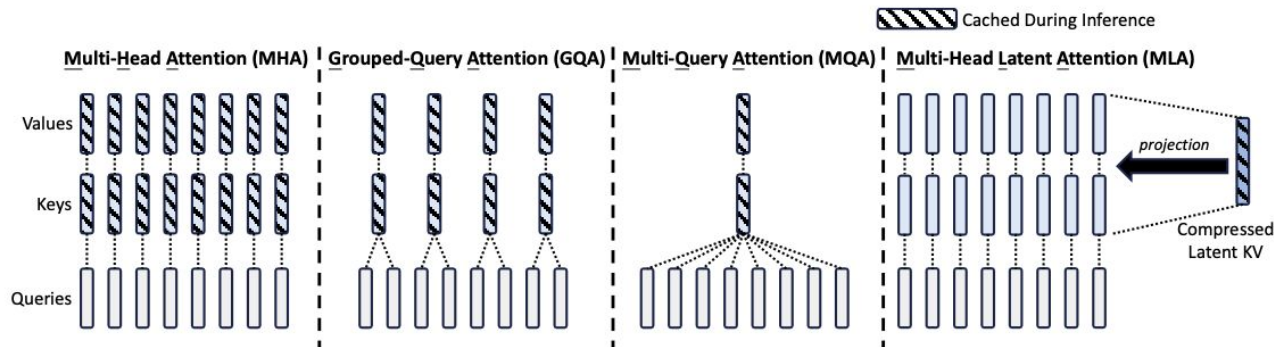


Figure 3 | Simplified illustration of Multi-Head Attention (MHA), Grouped-Query Attention (GQA), Multi-Query Attention (MQA), and Multi-head Latent Attention (MLA). Through jointly compressing the keys and values into a latent vector, MLA significantly reduces the KV cache during inference.

DeepSeek-V2 Architecture

Attention Mechanism	KV Cache per Token (# Element)	Capability
Multi-Head Attention (MHA)	$2n_h d_h l$	Strong
Grouped-Query Attention (GQA)	$2n_g d_h l$	Moderate
Multi-Query Attention (MQA)	$2d_h l$	Weak
MLA (Ours)	$(d_c + d_h^R)l \approx \frac{9}{2}d_h l$	Stronger

Table 1 | Comparison of the KV cache per token among different attention mechanisms. n_h denotes the number of attention heads, d_h denotes the dimension per attention head, l denotes the number of layers, n_g denotes the number of groups in GQA, and d_c and d_h^R denote the KV compression dimension and the per-head dimension of the decoupled queries and key in MLA, respectively. The amount of KV cache is measured by the number of elements, regardless of the storage precision. For DeepSeek-V2, d_c is set to $4d_h$ and d_h^R is set to $\frac{d_h}{2}$. So, its KV cache is equal to GQA with only 2.25 groups, but its performance is stronger than MHA.

DeepSeek-V2 MoE

Let $\mathbf{u}_t$ be the FFN input of the t -th token, we compute the FFN output $\mathbf{h}'_t$ as follows:

$$\mathbf{h}'_t = \mathbf{u}_t + \sum_{i=1}^{N_s} \text{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t} \text{FFN}_i^{(r)}(\mathbf{u}_t), \quad (20)$$

$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N_r\}, K_r), \\ 0, & \text{otherwise}, \end{cases} \quad (21)$$

$$s_{i,t} = \text{Softmax}_i(\mathbf{u}_t^T \mathbf{e}_i), \quad (22)$$

where N_s and N_r denote the numbers of shared experts and routed experts, respectively; $\text{FFN}_i^{(s)}(\cdot)$ and $\text{FFN}_i^{(r)}(\cdot)$ denote the i -th shared expert and the i -th routed expert, respectively; K_r denotes the number of activated routed experts; $g_{i,t}$ is the gate value for the i -th expert; $s_{i,t}$ is the token-to-expert affinity; $\mathbf{e}_i$ is the centroid of the i -th routed expert in this layer; and $\text{Topk}(\cdot, K)$ denotes the set comprising K highest scores among the affinity scores calculated for the t -th token and all routed experts.

DeepSeek-V2 MoE

- Device Limited routing
 - The routed experts will be distributed across multiple devices
 - Ensure that the target experts of each token will be distributed on at most M devices
- Expert Level Load Balancing - Auxiliary Loss
- Device Level Load Balancing
- Communication Balance Loss.

$$\mathcal{L}_{\text{ExpBal}} = \alpha_1 \sum_{i=1}^{N_r} f_i P_i,$$

$$f_i = \frac{N_r}{K_r T} \sum_{t=1}^T \mathbb{1}(\text{Token } t \text{ selects Expert } i),$$

$$P_i = \frac{1}{T} \sum_{t=1}^T s_{i,t},$$

$$\mathcal{L}_{\text{DevBal}} = \alpha_2 \sum_{i=1}^D f'_i P'_i,$$

$$f'_i = \frac{1}{|\mathcal{E}_i|} \sum_{j \in \mathcal{E}_i} f_j,$$

$$P'_i = \sum_{j \in \mathcal{E}_i} P_j,$$

$$\mathcal{L}_{\text{CommBal}} = \alpha_3 \sum_{i=1}^D f''_i P''_i,$$

$$f''_i = \frac{D}{MT} \sum_{t=1}^T \mathbb{1}(\text{Token } t \text{ is sent to Device } i),$$

$$P''_i = \sum_{j \in \mathcal{E}_i} P_j,$$

DeepSeek-V2 Performance

	Benchmark	# Shots	DeepSeek 67B Chat	Qwen 1.5 72B Chat	LLaMA3 70B Inst.	Mixtral 8x22B Inst.	DeepSeek-V2 Chat (SFT)	DeepSeek-V2 Chat (RL)
	Context Length	-	4K	32K	8K	64K	128K	128K
	Architecture	-	Dense	Dense	Dense	MoE	MoE	MoE
	# Activated Params	-	67B	72B	70B	39B	21B	21B
	# Total Params	-	67B	72B	70B	141B	236B	236B
English	TriviaQA	5-shot	81.5	79.6	69.1	80.0	85.4	86.7
	NaturalQuestions	5-shot	47.0	46.9	44.6	54.9	51.9	<u>53.4</u>
	MMLU	5-shot	71.1	76.2	80.3	77.8	<u>78.4</u>	77.8
	ARC-Easy	25-shot	96.6	96.8	96.9	97.1	<u>97.6</u>	98.1
	ARC-Challenge	25-shot	88.9	<u>91.7</u>	92.6	90.0	92.5	92.3
	BBH	3-shot	71.7	65.9	<u>80.1</u>	78.4	81.3	79.7
	AGIEval	0-shot	46.4	<u>62.8</u>	56.6	41.4	63.2	61.4
	IFEval	0-shot	55.5	57.3	79.7	<u>72.1</u>	64.1	63.8
Code	HumanEval	0-shot	73.8	68.9	76.2	75.0	<u>76.8</u>	81.1
	MBPP	3-shot	61.4	52.2	69.8	64.4	<u>70.4</u>	72.0
	CRUXEval-I-COT	2-shot	49.1	51.4	<u>61.1</u>	59.4	59.5	61.5
	CRUXEval-O-COT	2-shot	50.9	56.5	63.6	63.6	60.7	<u>63.0</u>
	LiveCodeBench	0-shot	18.3	18.8	<u>30.5</u>	25.0	28.7	32.5
Math	GSM8K	8-shot	84.1	81.9	93.2	87.9	90.8	<u>92.2</u>
	MATH	4-shot	32.6	40.6	48.5	49.8	<u>52.7</u>	53.9
	CMath	0-shot	80.3	82.8	79.2	75.1	<u>82.0</u>	<u>81.9</u>
Chinese	CLUEWSC	5-shot	78.5	90.1	85.4	75.8	88.6	89.9
	C-Eval	5-shot	65.2	82.2	67.9	60.0	<u>80.9</u>	78.0
	CMMLU	5-shot	67.8	82.9	70.7	61.0	<u>82.4</u>	81.6

Table 3 | Comparison among DeepSeek-V2 Chat (SFT), DeepSeek-V2 Chat (RL), and other representative open-source chat models. Regarding TriviaQA and NaturalQuestions, it is worth noting that chat models, such as LLaMA3 70B Instruct, might not strictly adhere to the format constraints typically specified in the few-shot setting. Consequently, this can lead to underestimation of certain models in our evaluation framework.

DeepSeek-V3

- Size: 671B
- MoE, so only 31B activated for each Token
 - Auxiliary-loss-free
- 128K context length
- Multi-head Latent Attention (MLA)
 - Compared with DeepSeek 67B,
 - Saves 42.5% of training costs
 - Reduces KV cache by 93.3%
 - Boosts Generation throughput to 5.76 times
- 14.8T tokens pre-training
- Requires only 2.788M H800 GPU hours for its full training
 - ~\$5.576M cost

DeepSeek-V3

- Size: 671B
- MoE, so only 31B activated for each Token
 - Auxiliary-loss-free
- 128K context length
- Multi-head Latent Attention (MLA)
 - Compared with DeepSeek 67B,
 - Saves 42.5% of training costs
 - Reduces KV cache by 93.3%
 - Boosts Generation throughput to 5.76 times
- 14.8T tokens pre-training
- Requires only 2.788M H800 GPU hours for its full training
 - ~\$5.576M cost

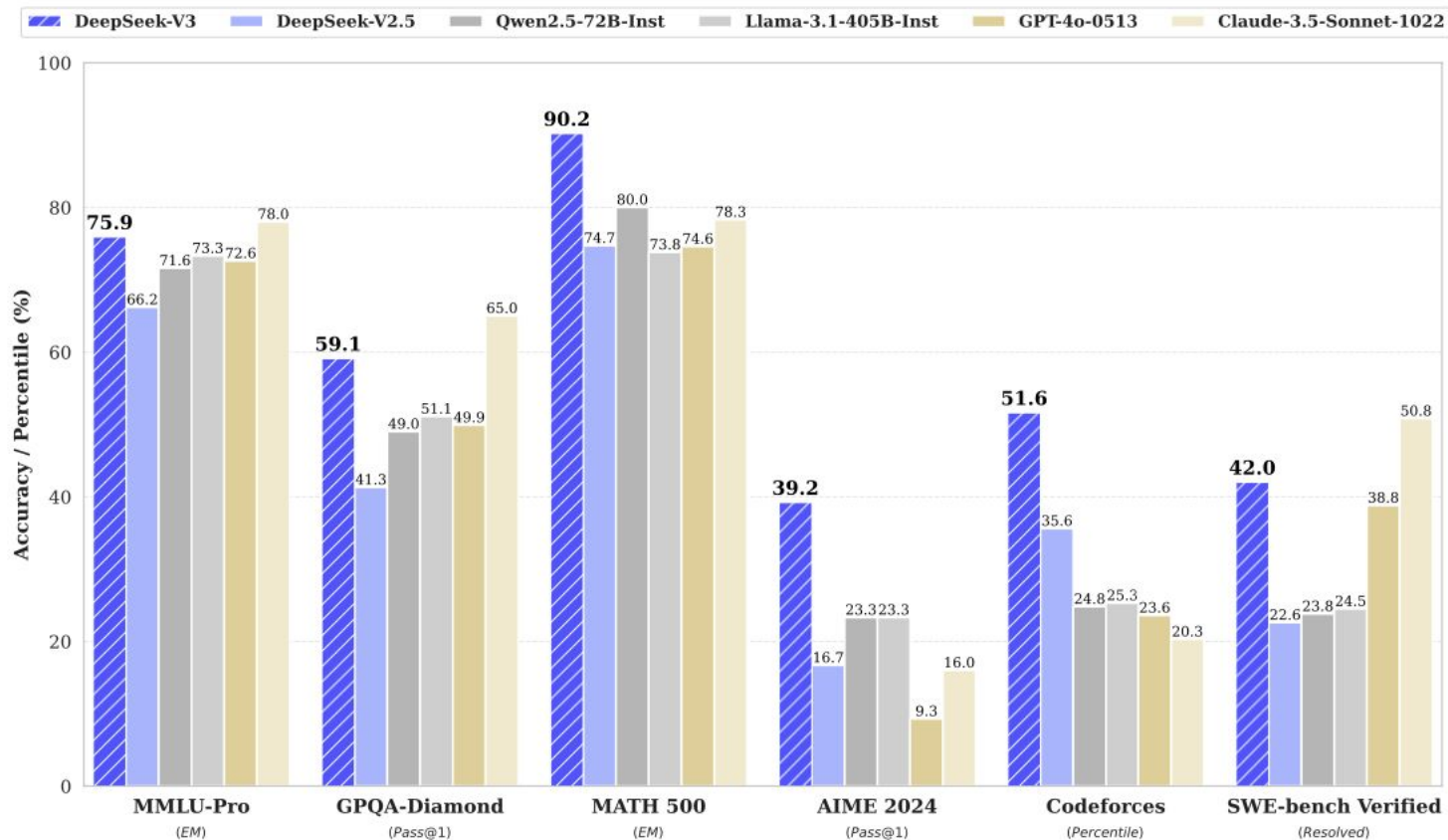


Figure 1 | Benchmark performance of DeepSeek-V3 and its counterparts.

DeepSeek-V3

- Size: 671B
- MoE, so only 31B activated for each Token
 - Auxiliary-loss-free
- 128K context length
- Multi-head Latent Attention (MLA)
 - Compared with DeepSeek 67B,
 - Saves 42.5% of training costs
 - Reduces KV cache by 93.3%
 - Boosts Generation throughput to 5.76 times
- 14.8T tokens pre-training
- Requires only 2.788M H800 GPU hours for its full training
 - ~\$5.576M cost

DeepSeek-V3: Key Innovations vs. V2

Auxiliary-Loss-Free Load Balancing. For MoE models, an unbalanced expert load will lead to routing collapse (Shazeer et al., 2017) and diminish computational efficiency in scenarios with expert parallelism. Conventional solutions usually rely on the auxiliary loss (Fedus et al., 2021; Lepikhin et al., 2021) to avoid unbalanced load. However, too large an auxiliary loss will impair the model performance (Wang et al., 2024a). To achieve a better trade-off between load balance and model performance, we pioneer an auxiliary-loss-free load balancing strategy (Wang et al., 2024a) to ensure load balance. To be specific, we introduce a bias term b_i for each expert and add it to the corresponding affinity scores $s_{i,t}$ to determine the top-K routing:

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Topk}(\{s_{j,t} + b_j | 1 \leq j \leq N_r\}, K_r), \\ 0, & \text{otherwise.} \end{cases} \quad (16)$$

Note that the bias term is only used for routing. The gating value, which will be multiplied with the FFN output, is still derived from the original affinity score $s_{i,t}$. During training, we keep monitoring the expert load on the whole batch of each training step. At the end of each step, we will decrease the bias term by γ if its corresponding expert is overloaded, and increase it by γ if its corresponding expert is underloaded, where γ is a hyper-parameter called bias update speed. Through the dynamic adjustment, DeepSeek-V3 keeps balanced expert load during training, and achieves better performance than models that encourage load balance through pure auxiliary losses.

DeepSeek-V3: Key Innovations vs. V2

MTP Training Objective. For each prediction depth, we compute a cross-entropy loss $\mathcal{L}_{\text{MTP}}^k$:

$$\mathcal{L}_{\text{MTP}}^k = \text{CrossEntropy}(P_{2+k:T+1}^k, t_{2+k:T+1}) = -\frac{1}{T} \sum_{i=2+k}^{T+1} \log P_i^k[t_i], \quad (24)$$

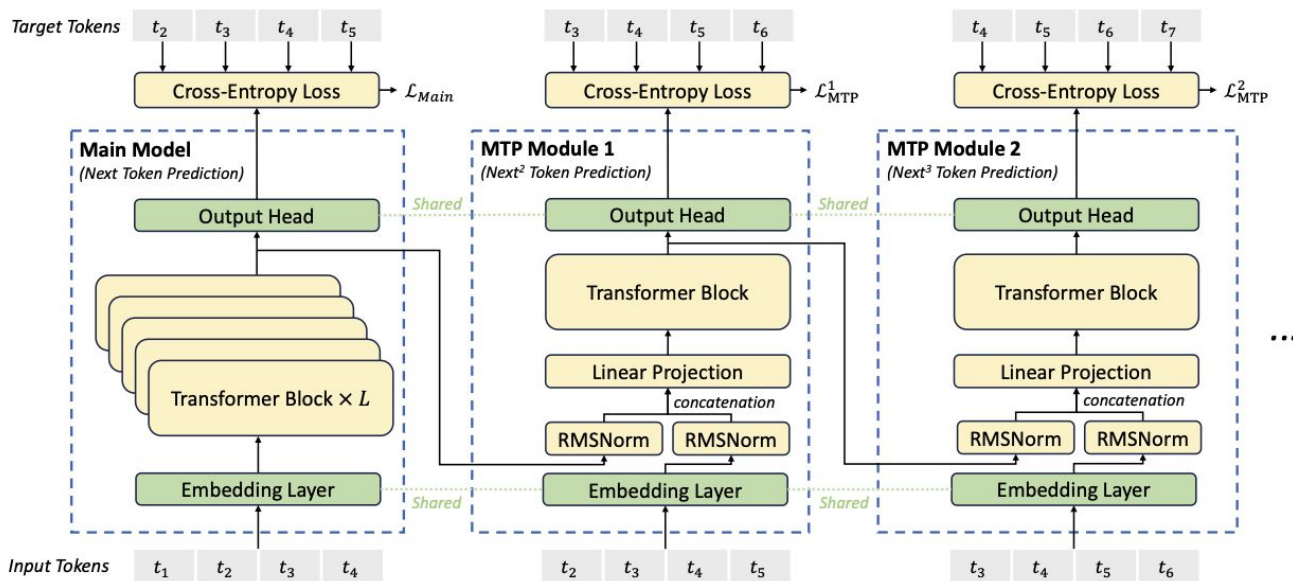


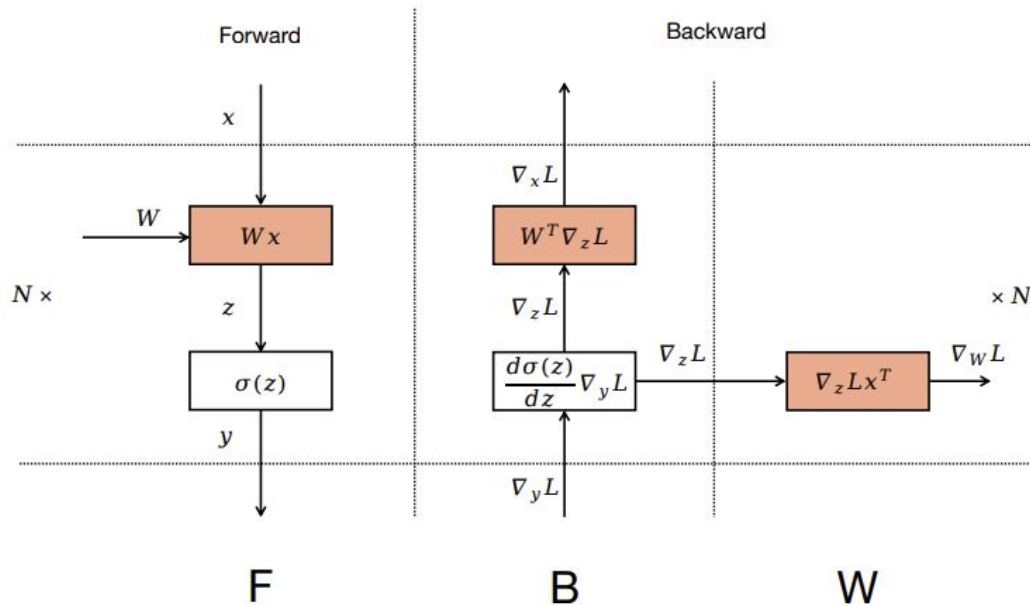
Figure 3 | Illustration of our Multi-Token Prediction (MTP) implementation. We keep the complete causal chain for the prediction of each token at each depth.

More Context on Pipeline Parallelism

- DP, TP and PP
 - Data Parallelism
 - copy the model to multiple GPUs, have data run through GPUs in Parallel
 - Tensor Parallelism
 - splits the matrix multiplication in one layer to several devices
 - Pipeline Parallelism
 - Zero, simple strategy to separate different layers of one Model Pipeline into different devices
 - The efficiency of its implementation relies heavily on the amount of device idle time referred to as pipeline bubbles
 - 1F1B, one-forward-one-backward
 - Early Scheduling the backward passes so reduce peak GPU memory requirement, same bubble rate

More Context on Pipeline Parallelism

- Zero Bubble
 - Further Separate out Backward to Input B(I) from Backward to Weight B(W)
 - B(I) is enough to start dispatch for next layer computation while B(W) can be computed in parallel



DeepSeek-V3: Key Innovations vs. V2

Zero Bubble Pipeline Parallelism

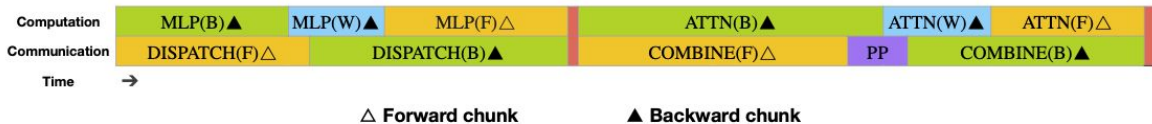


Figure 4 | Overlapping strategy for a pair of individual forward and backward chunks (the boundaries of the transformer blocks are not aligned). Orange denotes forward, green denotes "backward for input", blue denotes "backward for weights", purple denotes PP communication, and red denotes barriers. Both all-to-all and PP communication can be fully hidden.

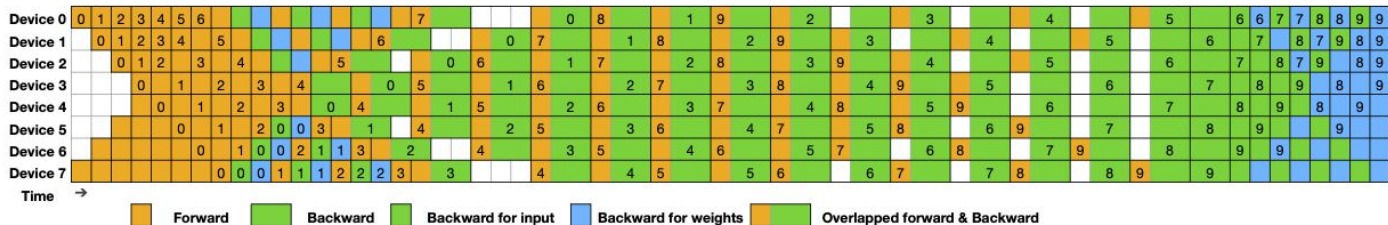


Figure 5 | Example DualPipe scheduling for 8 PP ranks and 20 micro-batches in two directions. The micro-batches in the reverse direction are symmetric to those in the forward direction, so we omit their batch ID for illustration simplicity. Two cells enclosed by a shared black border have mutually overlapped computation and communication.

DeepSeek-V3: Key Innovations vs. V2

Zero Bubble Pipeline Parallelism

Method	Bubble	Parameter	Activation
1F1B	$(PP - 1)(F + B)$	1×	PP
ZB1P	$(PP - 1)(F + B - 2W)$	1×	PP
DualPipe (Ours)	$(\frac{PP}{2} - 1)(F\&B + B - 3W)$	2×	$PP + 1$

Table 2 | Comparison of pipeline bubbles and memory usage across different pipeline parallel methods. F denotes the execution time of a forward chunk, B denotes the execution time of a full backward chunk, W denotes the execution time of a "backward for weights" chunk, and $F\&B$ denotes the execution time of two mutually overlapped forward and backward chunks.

DeepSeek-V3: Key Innovations vs. V2

Mixed Precision with FP8 Training

- GEMM FP8 General Matrix Multiplication

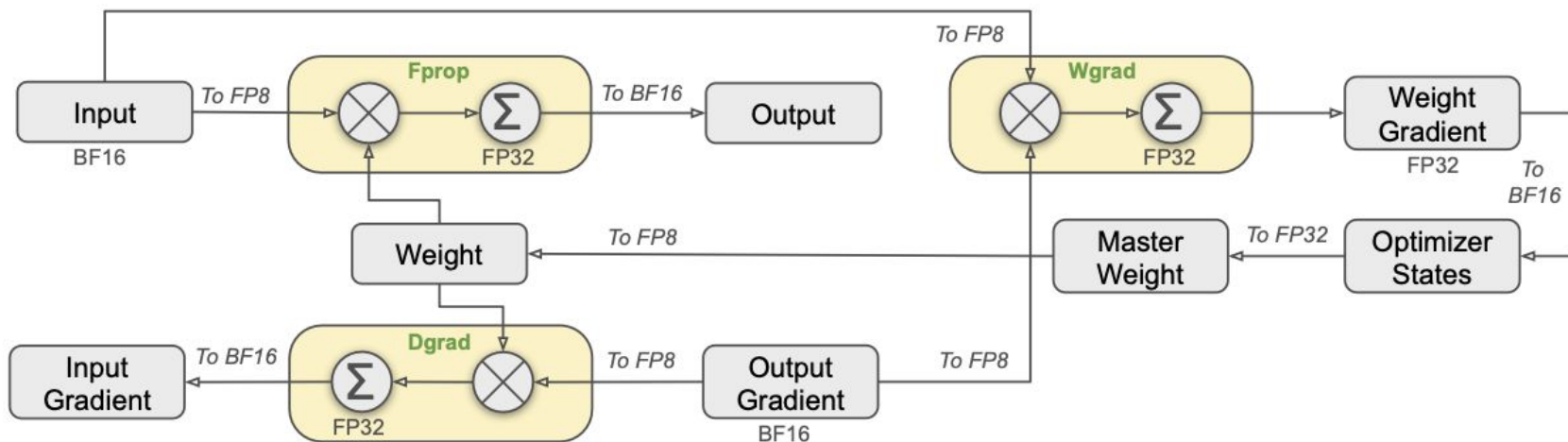


Figure 6 | The overall mixed precision framework with FP8 data format. For clarification, only the Linear operator is illustrated.

DeepSeek-R1

- Beat OpenAI o1
- The mode that causes WallStreet Panic and Nvidia Selloff!
- Demonstrates pure RL could achieves good reasoning capabilities
- Distillation also works
 - R1 into smaller open source models
- Combines all the good techniques from V3 and DeepSeek-Math

DeepSeek-R1 - Performance

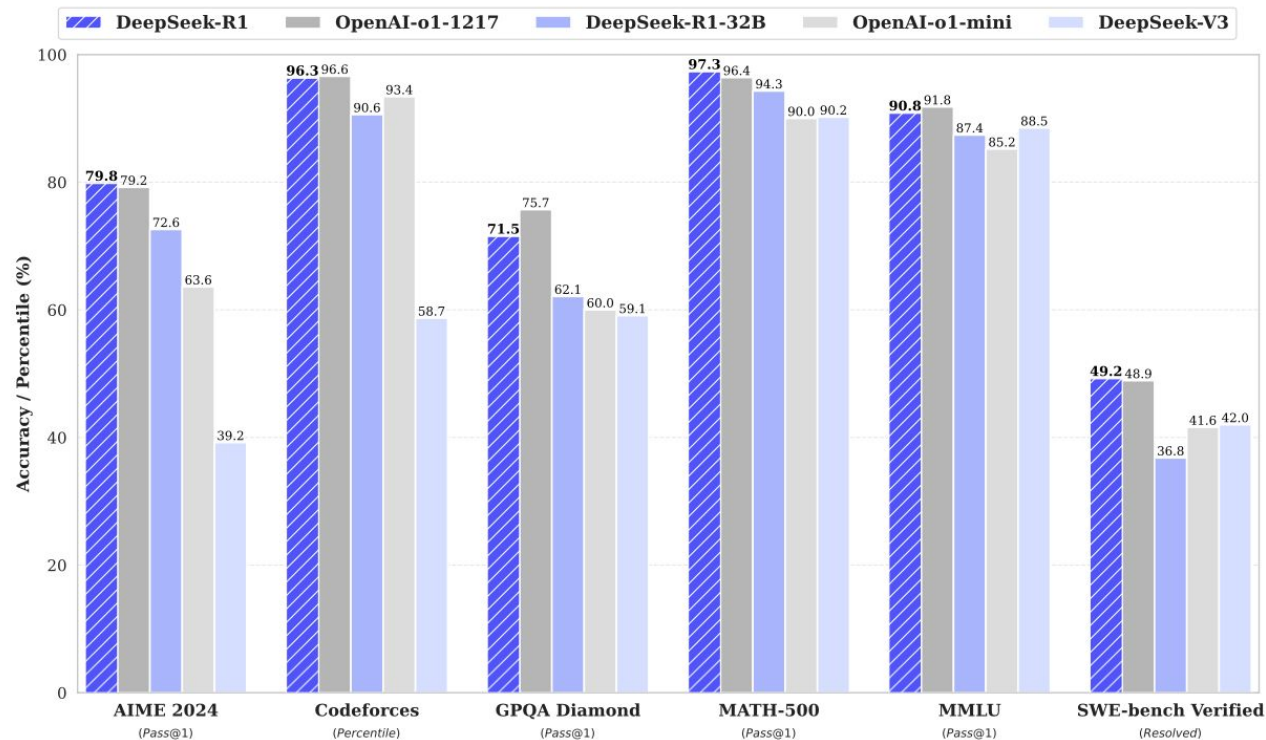


Figure 1 | Benchmark performance of DeepSeek-R1.

DeepSeek-R1 - RL With GRPO

Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q , GRPO samples a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_{θ} by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \varepsilon, 1 + \varepsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) \right), \quad (1)$$

$$\mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1, \quad (2)$$

where ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \dots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}. \quad (3)$$

DeepSeek-R1 - Rule Based Reward

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within `<think>` `</think>` and `<answer>` `</answer>` tags, respectively, i.e., `<think>` reasoning process here `</think>` `<answer>` answer here `</answer>`. User: **prompt**. Assistant:

Table 1 | Template for DeepSeek-R1-Zero. **prompt** will be replaced with the specific reasoning question during training.

The reward is the source of the training signal, which decides the optimization direction of RL. To train DeepSeek-R1-Zero, we adopt a rule-based reward system that mainly consists of two types of rewards:

- **Accuracy rewards:** The accuracy reward model evaluates whether the response is correct. For example, in the case of math problems with deterministic results, the model is required to provide the final answer in a specified format (e.g., within a box), enabling reliable rule-based verification of correctness. Similarly, for LeetCode problems, a compiler can be used to generate feedback based on predefined test cases.
- **Format rewards:** In addition to the accuracy reward model, we employ a format reward model that enforces the model to put its thinking process between '`<think>`' and '`</think>`'

DeepSeek-R1 - Pure RL works

Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	1820
OpenAI-o1-0912	74.4	83.3	94.8	77.3	63.4	1843
DeepSeek-R1-Zero	71.0	86.7	95.9	73.3	50.0	1444

Table 2 | Comparison of DeepSeek-R1-Zero and OpenAI o1 models on reasoning-related benchmarks.

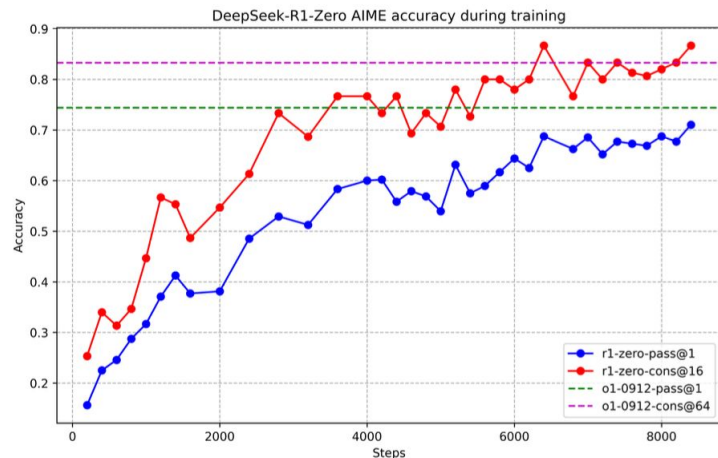


Figure 2 | AIME accuracy of DeepSeek-R1-Zero during training. For each question, we sample 16 responses and calculate the overall average accuracy to ensure a stable evaluation.

DeepSeek-R1 - Inference Time Scaling

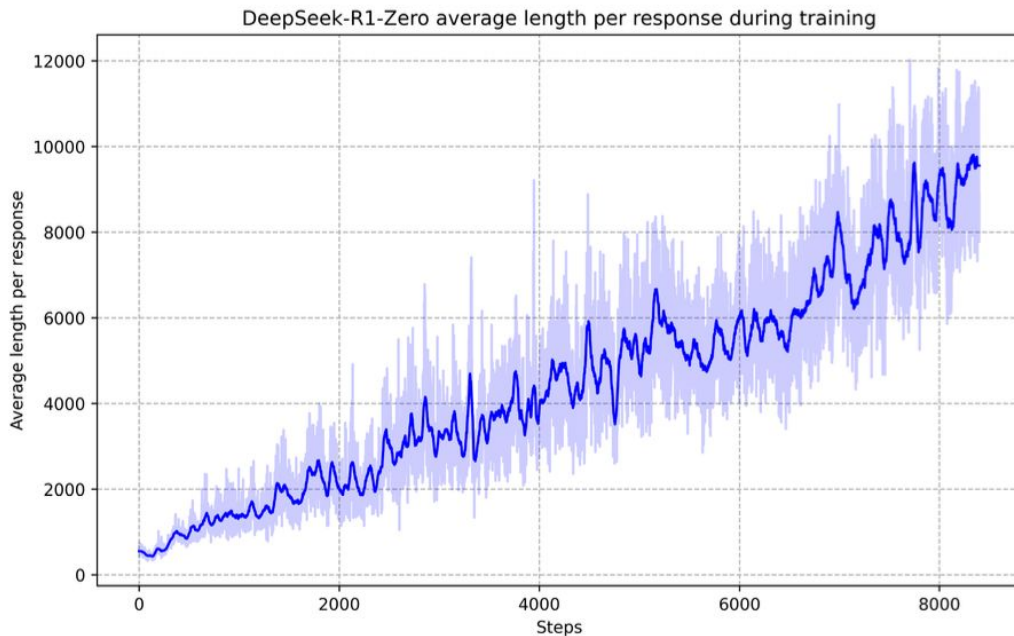


Figure 3 | The average response length of DeepSeek-R1-Zero on the training set during the RL process. DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time.

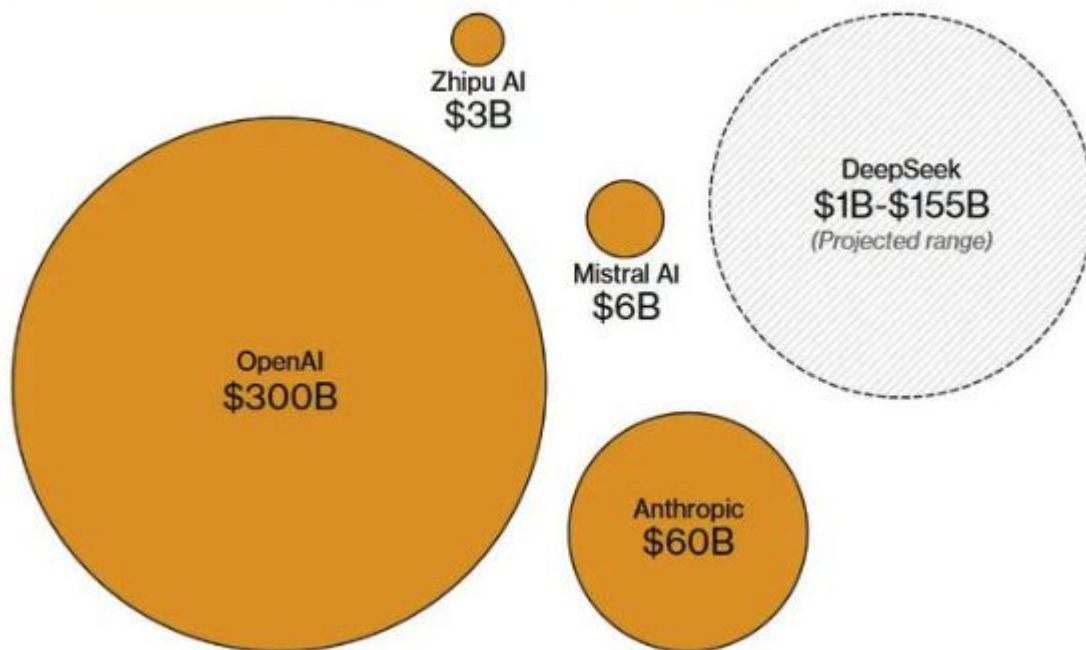
Summary

- An extremely talented group of people moving fast at iterating and innovations
- Stand on the shoulder of giants - OpenAI, LLaMa, etc.
- Major contributions and innovations to the large model community with publications
 - Pre-training data aggressive de-duplication
 - GPRO
 - FP8 mixed-precision training
 - Zero Bubble (DualPipe Pipeline Parallelism)
 - Pure RL strategy and Inference Time scaling
- Improved LLM training efficiency drastically, positively contributed to the whole AI industry

DeepSeek could be valued up to \$155B, per Bloomberg.

AI Boom

How DeepSeek's potential valuation stacks up against rivals



Source: Bloomberg

Reference

Guo, Daya, et al. "DeepSeek-Coder: When the Large Language Model Meets Programming--The Rise of Code Intelligence." *arXiv preprint arXiv:2401.14196* (2024).

Bi, Xiao, et al. "Deepseek llm: Scaling open-source language models with longtermism." *arXiv preprint arXiv:2401.02954* (2024).

Lu, Haoyu, et al. "Deepseek-vl: towards real-world vision-language understanding." *arXiv preprint arXiv:2403.05525* (2024).

Liu, Aixin, et al. "Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model." *arXiv preprint arXiv:2405.04434* (2024).

Liu, Aixin, et al. "Deepseek-v3 technical report." *arXiv preprint arXiv:2412.19437* (2024).

Guo, Daya, et al. "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning." *arXiv preprint arXiv:2501.12948* (2025).

Shao, Zhihong, et al. "Deepseekmath: Pushing the limits of mathematical reasoning in open language models." *arXiv preprint arXiv:2402.03300* (2024).

Guo, Daya, et al. "DeepSeek-Coder: When the Large Language Model Meets Programming--The Rise of Code Intelligence." *arXiv preprint arXiv:2401.14196* (2024).